

Lab 5 – Fusion Compiler – synteza i implementacja top-down

I. Oprogramowaniem Fusion Compiler

Fusion Compiler firmy Synopsys to oprogramowanie realizujące ujednoliconą syntezę logiczną i fizyczną od poziomu RTL do poziomu masek technologicznych w formacie GDSII. Fusion Compiler łączy tradycyjną syntezę logiczną z tworzeniem floorplanu, rozmieszczaniem komórek i połączeń (ang. place and route) i optymalizacją.

1. Uruchamianie oprogramowania Fusion Compiler

Fusion Compiler to oprogramowanie pracujące pod kontrolą systemów Unix/Linux (wybranych dystrybucji). Przed uruchomieniem programu Fusion Compiler konieczne jest ustawienie odpowiednich zmiennych i ścieżek systemowych. W tym celu należy wykonać przygotowany uprzednio skrypt, wywołując z linii komend następujące polecenie:

```
add-synopsys-IMP-all
```

Następnie należy przejść od katalogu roboczego i uruchomić program Fusion Compiler wpisując w linii komend polecenie:

```
fc_shell
```

Zaleca się aby przed uruchomieniem programu Fusion Compiler uruchomić w terminalu nową powłokę poprzez wykonanie polecenia:

```
bash
```

W razie wystąpienia problemów we współpracy z omawianym programem powyższy zabieg ułatwi powrót do pierwotnego środowiska pracy.

Samo polecenie `fc_shell` uruchamia program Fusion Compiler w trybie terminalowym. Aby uruchomić ten program w trybie graficznym konieczne jest użycie przełącznika linii komend `-gui`, jak to pokazano poniżej:

```
fc_shell -gui &
```

Interfejs graficzny programu można także włączyć i wyłączyć z `fc_shell` używając odpowiednio instrukcji:

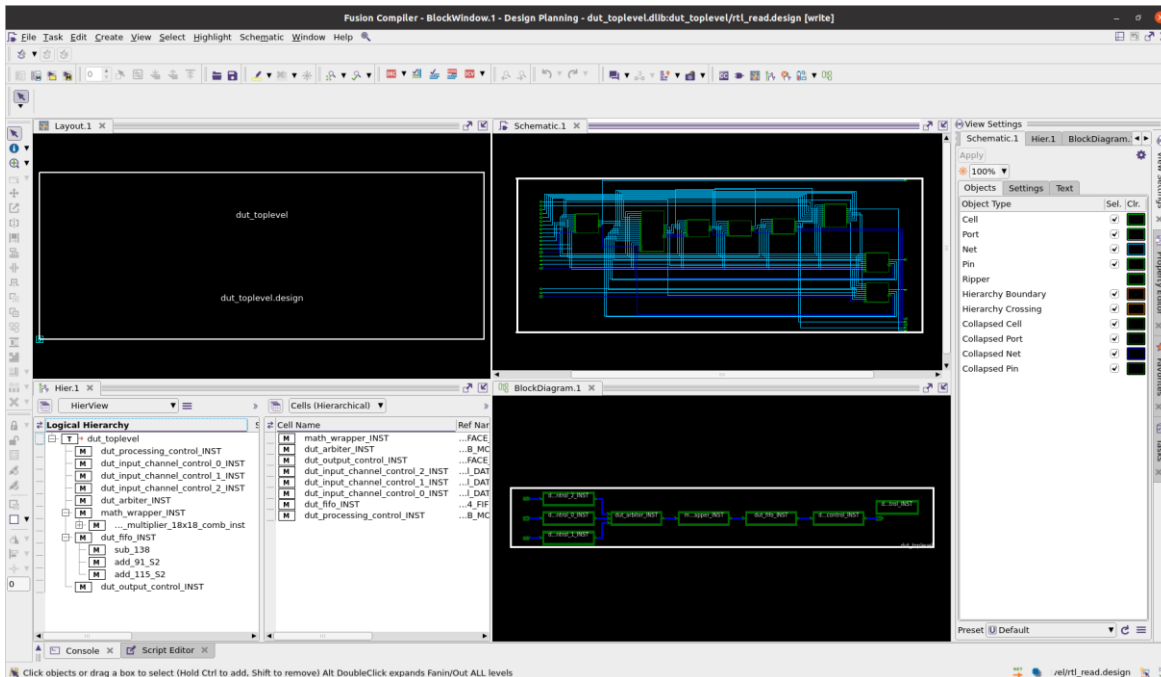
```
gui_start  
gui_stop
```

Program Fusion Compiler pozwala także na wykonanie zaraz po uruchomieniu wskazanego skryptu w języku TCL. W takim przypadku program należy wywołać w jednym z dwóch poniższych sposobów:

```
fc_shell -f <ścieżka_do_skryptu>/<nazwa_skryptu>.tcl  
fc_shell -f <ścieżka_do_skryptu>/<nazwa_skryptu>.tcl -gui
```

2. Interfejs graficzny programu Fusion Compiler

Widok głównego okna programu Fusion Compiler pokazano na rys. 1. U góry okna widoczny jest pasek menu i pasek narzędzi. Poniżej otwartych jest kilka okien podrzędnych, m. in. okno przedstawiające layout układu (tu zawiera tylko jedną komórkę), okna zawierające schemat logiczny oraz schemat blokowy układu a także okno prezentujące hierarchię projektu. Po prawej stronie głównego okna znajduje się pasek zakładek.



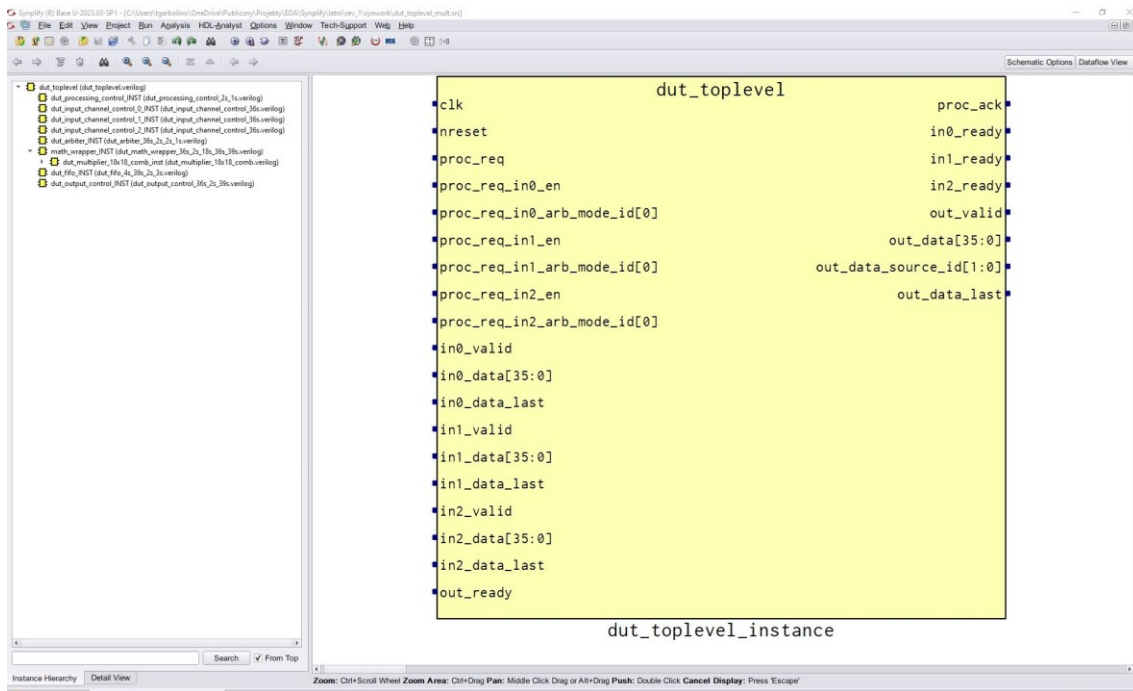
Rys. 1. Widok głównego okna programu Fusion Compiler

Na rys. 1 widoczna jest akurat zakładka ustawień wyświetlania.

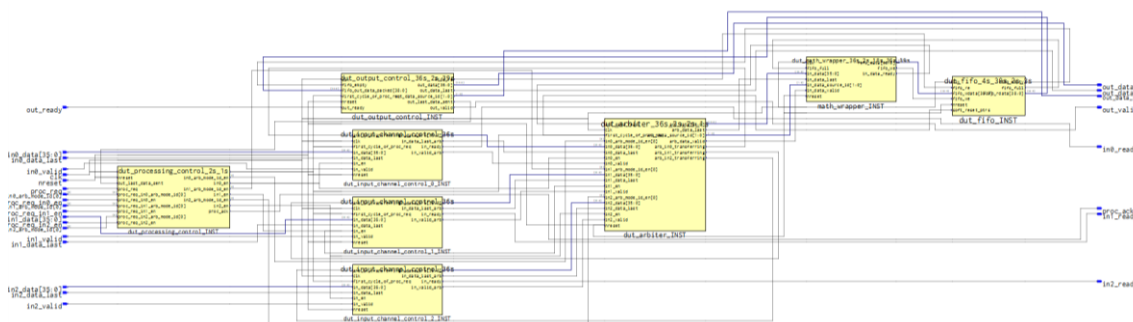
II. Zapoznanie się z realizowanym projektem

W trakcie zajęć laboratoryjnych zostanie wykonana synteza i implementacja modelu układu DUT (ang. Design Under Test) udostępnionego przez firmę Intel. Model został napisany całkowicie w języku opisu sprzętu SystemVerilog HDL. Interfejs i strukturę hierarchiczną modelu pokazano na rys. 3 (jest to widok jednego z okien programu Synplify firmy Synopsys).

Wewnętrzną strukturę blokową modelu pokazano z kolei na rys. 2 (ten schemat także został wygenerowany przez program Synplify). DUT zawiera trzy kanały wejściowe oraz kanał wyjściowy wraz z układami zarządzającymi transmisją danych i kolejką FIFO, moduł arytmetyczny, układ arbitrażu a także układ sterujący całością przetwarzania danych.



Rys. 3. Struktura hierarchiczna i interfejs modelu DUT



Rys. 2. Schemat blokowy modelu DUT

III. Przebieg zajęć laboratoryjnych

W ramach zajęć laboratoryjnych zostanie wykonanych kilka ćwiczeń prezentujących przebieg syntezy logicznej i implementacji układu cyfrowego do postaci układu ASIC.

Każde ćwiczenie rozpoczynamy od przejścia do katalogu:

```
<lab_root_path>/asic_lab6/lab/tasks/task<task_no>/work
```

gdzie *<lab_root_path>* to ścieżka do miejsca, w którym umieszczono katalog *asic_lab6* wraz z podkatalogami, natomiast *<task_no>* to numer aktualnie wykonywanego zadania.

W katalogu *work* znajduje się skrypt *Makefile* zawierający listę operacji dla polecenia systemowego *make*. Wpisanie w linii komend polecenia:

```
make clean
```

powoduje doprowadzenie katalogów związanych z danym ćwiczeniem do stanu początkowego. Zaleca się wykonanie powyższego polecenia przed rozpoczęciem i powtórzeniem każdego ćwiczenia. Z kolei polecenie:

```
make run
```

powoduje uruchomienie skryptu TCL tworzonego przez użytkownika w trakcie zajęć.

Każde z ćwiczeń należy rozpocząć od wykonania w *fc_shell* skryptów inicjujących odpowiednie zmienne potrzebne w dalszej realizacji ćwiczenia:

```
source -echo ../../../../setup/main_setup.tcl
source -echo ${SetupDir}/design_setup.tcl
```

Ćwiczenie 0

Zadaniem tego ćwiczenia jest wyłącznie utworzenie biblioteki projektowej oraz kompilacja i elaboracja plików źródłowych projektu. Z poziomu konsoli należy jedynie wykonać sekwencję poleceń:

```
make clean
make run
```

Ćwiczenie 1

To ćwiczenie jest w do pewnego stopnia krótką powtórką z części poprzednich ćwiczeń, która była poświęcona tworzeniu floorplanu układu oraz sieci zasilania. Ćwiczenie zostało podzielone na trzy następujące etapy:

- automatyczne stworzenie floorplanu o dobranych parametrach,
- manualne stworzenie floorplanu,
- utworzenie sieci zasilania.

Na potrzeby realizacji każdego z w/w etapów przygotowano odrębny skrypt, odpowiednio: `tasklaf.tcl`, `tasklmf.tcl` oraz `tasklpg.tcl`. Skrypty te należy uruchamiać kolejno, używając odpowiedniego argumentu instrukcji `make`: `run_af`, `run_mf` oraz `run_pg`. Przed uruchomieniem pierwszego skryptu należy koniecznie wykonać polecenie `make clean`. Ponadto, w trakcie ćwiczenia wykorzystywane są także dwa dodatkowe skrypty `insert_special_physical_cells.tcl` oraz `create_pg_network.tcl`, które wywoływane są przez niektóre główne skrypty.

Funkcja automatycznego generowania dostrojonego floorplanu została tu przede wszystkim użyta do doboru odpowiednich parametrów floorplanu – przede wszystkim współczynnika `core_utilization`. Następnie wykorzystano funkcję ręcznego tworzenia floorplanu do przygotowania tzw. czystego floorplanu na potrzeby późniejszych operacji rozmieszczenia komórek (ang. *placement*) i optymalizacji.

Ręcznie utworzony floorplan zostaje uzupełniony o sieć połączeń zasilania i masy, której powstanie przebiega w następujących krokach:

- usunięcie wszystkich danych z ewentualnych poprzednich prób tworzenia sieci zasilania i masy,
- zdefiniowanie nazw linii zasilania i masy,
- poprowadzenie szyn bezpośrednio zasilających komórki standardowe (ang. *rails*),
- stworzenie pionowych i poziomych linii (tzw. pasków) zasilania i masy (ang. *power and ground straps*),
- wygenerowanie pierścieni zasilania i masy (ang. *power and ground rings*), biegnących wokół rdzenia układu,
- odpowiednie połączenie elementów sieci zasilania i masy przelotkami (ang. *via*),
- podłączenie pinów zasilania i masy komórek standardowych i bloków do sieci zasilającej,
- weryfikacja zgodności powstałej sieci zasilającej z regułami projektowymi.

Widok layoutu po wykonaniu każdego z głównych skryptów można zobaczyć uruchamiając program Fusion Compiler w trybie graficznym i korzystając np. z poleceń menu: `File`→`Open Library...` oraz `File`→`Open block...`. W przypadku końcowego layoutu, aby np. zobaczyć szyny zasilania komórek standardowych, konieczne może być wyłączenie widoczności niektórych warstw i niektórych elementów. Należy w tym celu skorzystać z zakładki `View Settings`, znajdującej się po prawej stronie głównego okna programu.

Zadania do wykonania

Przeanalizuj skrypty oraz widoki layoutów powiązane w niniejszym ćwiczeniu i odpowiedz na poniższe pytania. Swoje odpowiedzi zamieść w sprawozdaniu.

- Jakie są wartości zastosowanych parametrów `core_utilization` oraz `core_offset`?
- Jaki jest stosunek wysokości do szerokości rdzenia układu DUT?
- Na których krawędziach układu zostały umieszczone funkcjonalne piny we/wy? W odpowiedzi użyj określeń: lewa, prawa, górna, dolna.
- Na której krawędzi układu został umieszczony pin sygnału zegarowego? W odpowiedzi użyj określeń: lewa, prawa, górna, dolna.
- Do jakich warstw metalizacji zostały przypisane piny we/wy układu?
- Do jakich warstw metalizacji został przypisany pin sygnału zegarowego?
- Jaka warstwa metalizacji została użyta do poprowadzenia szyn zasilania komórek standardowych?
- Które warstwy metalizacji zostały wykorzystane do utworzenia pierścieni zasilania i masy?
- Które warstwy metalizacji zostały wykorzystane do poprowadzenia linii (tzw. pasków) zasilania?
- Które warstwy metalizacji zostały wykorzystane do poprowadzenia linii (tzw. pasków) masy?

W odpowiedziach na pytania dotyczące sieci zasilającej należy wskazać kierunek prowadzenia linii zasilania/masy w danej warstwie (pionowy lub poziomy).

Oprócz odpowiedzi na powyższe pytania w sprawozdaniu należy zamieścić zrzuty ekranu pokazujące layout układu DUT po każdym etapie tworzenia floorplanu. Ponadto w sprawozdaniu powinien znaleźć się także widok layoutu, na którym spośród elementów sieci zasilania widoczne będą wyłącznie szyny masy i zasilania komórek standardowych.

Ćwiczenie 2

Operacją następującą po przygotowaniu floorplanu układu i uzupełnieniu go o sieć zasilającą jest optymalne rozmieszczenie komórek standardowych. Od jakości jej realizacji bardzo silnie zależy poprawne wykonanie dalszych etapów projektu układu ASIC.

Tradycyjnie ćwiczenie zaczynamy od wykonania skryptów konfiguracyjnych, otwarcia biblioteki i stworzenia nowego bloku projektowego.

```
#### Sourcing main setup script
source -echo ../../../../setup/main_setup.tcl
#### Sourcing project specific setup script
source -echo ${SetupDir}/design_setup.tcl

# Open the design library
open_lib ${ResultsDir}/${DesignLibrary}

# Copy and open block
copy_block -from ${DesignName}/final_floorplan \
-to ${DesignName}/placement_and_optimization
open_block ${DesignName}/placement_and_optimization
```

Sposób rozmieszczenia komórek kontrolowany jest przez szereg opcji programu Fusion Compiler, których nazwy mają ogólną postać **place.***. Te wykorzystywane w niniejszym ćwiczeniu zebrano w skrypcie `placement_options.tcl`, który należy wykonać przed przejściem do kolejnych operacji.

```
source -echo placement_options.tcl
```

Operacja rozmieszczania komórek standardowych normalnie wymaga dostarczenia pliku definiującego strukturę i parametry ścieżek sterująco-obsługowych (ang. scandef file). Aby kontynuować pracę programu bez tego pliku konieczne jest ustawienie opcji `place.coarse.continue_on_missing_scandef`.

Dwie kolejne opcje zawarte w skrypcie `set_placement_options.tcl` decydują o stopniu optymalizacji rozmieszczania komórek standardowych, m. in. pod kątem unikania tzw. zatorów (ang. congestion). Zadaniem następnych komend zawartych w w/w skrypcie jest wskazanie zestawu komórek bibliotecznych wykorzystywanych w dalszych operacjach oraz odpowiednie sformatowanie nazw ich instancji.

W celu przygotowania projektu do operacji rozmieszczania komórek standardowych (ang. placement) oraz optymalizacji należy teraz wykonać instrukcję:

```
compile_fusion -to initial_opto
```

Na widoku layoutu układu można zauważyć, że w fazie `initial_opto` instrukcji `compile_fusion` jest już wykonywana wstępna operacja rozmieszczenia komórek (może być konieczne wyłączenie widoczności linii sieci zasilającej układ).

Rozmieszczenie komórek standardowych możemy przeprowadzić także w klasyczny sposób. Rozpoczynamy od usunięcia (zresetowania) aktualnego rozmieszczenia komórek:

```
reset_placement
```

Następnie dokonujemy zgrubnego rozmieszczenia komórek, sterując tą operacją za pomocą szeregu różnych opcji, kładących np. nacisk na parametry czasowe układu (ang. timing-driven placement), unikanie tzw. zatorów (ang. congestion-driven placement), itp.

```
create_placement \  
    -timing_driven \  
    -congestion \  
    -congestion_effort medium \  
    -buffering_aware_timing_driven
```

Zauważmy, że po tej operacji komórki standardowe zostały co prawda rozmieszczone wewnątrz rdzenia układu, ale w nieprecyzyjny sposób. Obszary komórek się wzajemnie nakładają, w związku z czym konieczne jest przesunięcie komórek do dozwolonych pozycji (ang. placement legalization) za pomocą instrukcji:

```
legalize_placement
```

Po wykonaniu powyższych operacji uruchamiamy optymalizację rozmieszczenia komórek standardowych instrukcją:

```
place_opt
```

Aby sprawdzić poprawność i parametry layoutu z rozmieszczonymi komórkami standardowymi należy wykonać zestaw poniższych komend i procedur:

```
check_legality  
report_congestion  
report_utilization  
generateReports classic_placement
```

Zadanie projektowe do samodzielnego wykonania

Napisz procedurę `analyzeDesign`, która przekieruje wyniki działania instrukcji `check_legality`, `report_congestion`, `report_utilization` do odpowiednich plików w katalogu zawierającym raporty z operacji. Np. wynik działania polecenia `check_legality` powinien zostać zapisany w pliku `classic_placement_check_legality.rpt`, gdzie `classic_placement` to nazwa bieżącego etapu projektu.

Należy mieć na uwadze, że instrukcja `compile_fusion` przeprowadzi operację rozmieszczania komórek i optymalizacji w ostatnich etapach swojego działania. Więc w praktyce nie ma potrzeby oddzielnego rozmieszczania komórek w klasyczny sposób, tak jak to zostało wykonane powyżej. Sprawdźmy to wykonując następujące instrukcje i obserwując layout układu:

```
reset_placement
compile_fusion -to final_opto
```

Podobnie jak poprzednio należy sprawdzić poprawność i parametry uzyskanego layoutu (ten etap projektu można nazwać np. `unified_placement`).

W niektórych przypadkach konieczne jest uniemożliwienie narzędziu EDA rozmieszczenia komórek w pewnych obszarach rdzenia układu. Obszar taki definiujemy instrukcją `create_placement_blockage`. Poniżej podano przykładowy zestaw instrukcji definiujący taki zabroniony obszar w układzie DUT. W tym obszarze nie powinny się znaleźć komórki standardowe.

```
create_placement_blockage \
    -boundary {{22.9780 21.6000} \
              {22.9780 27.4000} \
              {31.8940 27.4000} \
              {31.8940 21.6000}} \
    -type hard

check_legality
legalize_placement -incremental
```

Po zakończeniu operacji rozmieszczania komórek standardowych (ang. `placement`) i optymalizacji układu konieczne jest jeszcze podłączenie wszystkich komórek do sieci zasilania za pomocą instrukcji:

```
connect_pg_net -net VDD [get_pins -hierarchical */VDD]
connect_pg_net -net VSS [get_pins -hierarchical */VSS]
```

W tym miejscu pozostaje jeszcze sprawdzenie poprawności i parametrów otrzymanego layoutu (ten etap projektu można nazwać np. `placement_with_blockage`).

Obecne ćwiczenie jak i następne kończymy wydając poniższy zestaw komend:

```
stop_gui
get_blocks -all
list_blocks
save_block
save_lib
close_blocks
close_lib
exit
```

Sprawozdanie

W części sprawozdania dotyczącej ćwiczenia nr 2 należy zamieścić:

- treść opracowanej procedury w języku TCL
- zrzuty ekranów pokazujących layout układu (bez widoku sieci zasilających) na poszczególnych etapach projektu (*classic_placement*, *unified_placement*, *placement_with_blockage*),

- zestawienie najważniejszych informacji otrzymanych z wygenerowanych raportów i ewentualnie tzw. logów.

Ćwiczenie 3

Do syntezy drzewa sygnału zegarowego (ang. Clock Tree Synthesis - CTS) możemy przystąpić po poprawnym zakończeniu etapu rozmieszczania komórek standardowych (ang. placement) i optymalizacji układu. Ponadto kluczowe parametry układu, takie jak m. in. częstotliwość pracy, pobór mocy i zajmowana powierzchnia (ang. performance, power, area – PPA) muszą spełniać przyjęte w projekcie wymagania.

Tę część zajęć laboratoryjnych rozpoczynamy poprzez utworzenie w oparciu o rezultaty poprzedniego ćwiczenia nowego bloku projektowego o nazwie `clock_tree_synthesis`. Przebieg syntezy drzewa sygnału zegarowego sterowany jest przez szereg opcji programu Fussion Compiler, których nazwy mają ogólną postać `cts.*`. Za ustawienie odpowiednich wartości tych opcji odpowiada skrypt `set_cts_options.tcl`. Jedna z nich określa np. maksymalną wartość współczynnika rozprętu (ang. fanout). Inne odpowiedzialne są m. in. za ułatwienie prowadzenia połączeń w układzie.

Jeżeli źródło sygnału zegarowego nie jest powiązane z żadnym padem we/wy, konieczne jest wskazanie typu komórki sterującej pinem/portem sygnału zegarowego.

```
set_driving_cell -lib_cell ${CellPrefix}_INV_20 \
    [get_ports ${ClockName}]
```

Należy także wybrać komórki, które zostaną użyte w trakcie syntezy drzewa sygnału zegarowego. W praktyce wykorzystuje się wzmacniacze (bufory) logiczne i inwertery logiczne (bramki NOT) o kilku różnych obciążalnościach (ang. driving strength).

Zadanie projektowe do samodzielnego wykonania

Napisz krótki skrypt w języku TCL o nazwie `cts_cell_usage.tcl`, wybierający komórki wykorzystywane w syntezie drzewa sygnału zegarowego. Przykład komendy realizującej to zadanie dla negacji o obciążalności 1 i bufora o obciążalności 2 podano poniżej:

```
set_lib_cell_purpose -include cts \
    { */SAEDRVT14_INV_1 */SAEDRVT14_BUF_2 }
```

W trakcie operacji CTS należy użyć negacji o obciążalnościach: 1, 2, 4, 8, 16, 20 oraz buforów o obciążalnościach: 2, 4, 6, 8, 16, 20. Zaleca się wykorzystanie instrukcji `foreach`. Opracowany skrypt umieść w katalogu `scripts` ćwiczenia nr 3

Sygnał zegarowy charakteryzuje się najwyższą w układzie częstotliwością przełączania. W związku z tym ścieżki sygnału zegarowego muszą zostać właściwie i starannie poprowadzone aby uniknąć wzajemnych zakłóceń pomiędzy sygnałem zegarowym a pozostałymi sygnałami w układzie. Zazwyczaj w przypadku sygnału zegarowego stosowane są niestandardowe reguły prowadzenia ścieżek (ang. Nondefault Routing rules – NDR). Przykład definicji i wykorzystania takich niestandardowych reguł zawiera skrypt `clock_ndr.tcl`. Zauważmy np., że w przypadku sygnału zegarowego stosowane są ścieżki o większych szerokościach i większych wzajemnych odstępach.

Przed przystąpieniem do syntezy drzewa sygnału zegarowego wskazane jest także zdefiniowanie pożądanej maksymalnej wartości „skew” dla wszystkich sygnałów zegarowych w projekcie:

```
set_clock_tree_options -clocks [all_clocks] \
    -target_skew 0.05
```

Informację o sygnałach zegarowych można uzyskać korzystając z polecenia:


```
get_clocks
```

Proces syntezy drzewa sygnału zegarowego składa się z trzech etapów:

- syntezy drzew sygnału zegarowego i ich optymalizacji,
- szczegółowego prowadzenia ścieżek/sieci sygnału zegarowego,
- optymalizacji całego drzewa i umieszczenia połączeń sygnałowego w przeznaczonych do tego celu kanałach.

Wszystkie trzy powyższe etapy można zrealizować komendą:

```
clock_opt
```

Wyświetlenie listy etapów syntezy drzewa sygnałowego zapewnia instrukcja:

```
clock_opt -list_only
```

Samą syntezę drzewa sygnału zegarowego realizuje polecenie:

```
clock_opt -to build_clock
```

Po jego wykonaniu drzewa sygnału zegarowego są rozprowadzone po lauyoucie układu.

Za szczegółowe prowadzenie ścieżek sygnału zegarowego zgodnie z regułami przez NDR odpowiada instrukcja:

```
clock_opt -from build_clock -to route_clock
```

Po jej wykonaniu można np. zauważyć, że ścieżki sygnału zegarowego są wyraźnie szersze od pozostałych połączeń.

Natomiast po wykonaniu instrukcji:

```
clock_opt -to final_opto
```

otrzymujemy layout układu ze zoptymalizowanymi drzewami sygnału zegarowego i zgrubnie poprowadzonymi pozostałymi połączeniami.

W celu zabezpieczenia sieci sygnału zegarowego przed zakłóceniami niezbędne jest poprowadzenie dodatkowych ekranujących linii masy. Przed tą operacją konieczne jest usunięcie ścieżek pozostałych sygnałów z układu (oprócz sieci sygnału zegarowego).

```
remove_routes -global_route
```

Wygenerowanie linii ekranujących zapewniają następujące dwa polecenia:

```
set clock_nets [get_nets -hierarchical \  
    -filter "net_type == clock"]  
create_shields -nets ${clock_nets} \  
    -with_ground VSS \  
    -preferred_direction_only true \  
    -align_to_shape_end true
```

Po ich wykonaniu można zaobserwować pojawienie się pionowych linii masy otaczających wybrane fragmenty linii sygnału zegarowego.

Po tej operacji zalecane jest ponowne podłączenie sieci masy i zasilania do komórek standardowych. W tym miejscu wskazane jest także sprawdzenie poprawności i parametrów otrzymanego layoutu, co robimy podobnie jak w poprzednim ćwiczeniu (można też wykorzystać napisaną procedurę `analyzeDesign`). Należy przyjąć, że nazwa bieżącego etapu projektowego to `cts`.

Na zakończenie podświetlimy otrzymane drzewo sygnału zegarowego korzystając z menu *Highlight → Color by → Clock Tree*.

Sprawozdanie

W części sprawozdania poświęconej ćwiczeniu nr 3 należy zamieścić zrzuty ekranu otrzymane po zakończeniu każdego z etapów syntezy drzewa sygnału zegarowego a także widok layoutu układu z podświetlonym drzewem sygnału zegarowego. Proszę również zamieścić informację o najważniejszych parametrach układu po wykonaniu operacji CTS.

Ćwiczenie 4

Po utworzeniu drzewa sygnału zegarowego układ jest gotowy do przeprowadzenia operacji prowadzenia połączeń pozostałych sygnałów (ang. routing). Wcześniej konieczne jest jednak ustawienie opcji kontrolujących jej przebieg. Ich nazwy mają postać **route.***. Zauważmy, że większość z tych opcji kładzie nacisk na optymalizację parametrów czasowych układu (ang. timing-driven routing).

Do ograniczenia warstw metalizacji, w których mogą być prowadzenia połączenia służy polecenie `set_ignored_layers`. Natomiast aby sprawdzić te restrykcje należy wykonać polecenie `report_ignored_layers`.

W pewnych przypadkach może zaistnieć potrzeba wyłączenia pewnego obszaru z prowadzenia ścieżek. Trzeba w tym celu wykorzystać komendę `create_routing_blockage`. W skrypcie `task4.tcl` podano przykład zdefiniowania takiego zabronionego obszaru dla warstw M2, M3 i M4, ale wyłącznie dla sieci sygnałowych. W zawiązku z tym np. linie sygnału zegarowego i ekranujące linie masy mogą nadal być prowadzone w tym obszarze. Znaki „?” należy zastąpić koordynatami narożników obszaru zabronionego. W tym konkretnym przypadku zaleca się wyspecyfikowanie tego obszaru tak, żeby pokrywał się z obszarem zakazany zdefiniowanym w ćwiczeniu 2.

Informacje o liczbie połączeń w projekcie i warstwach metalizacji dostępnych dla operacji prowadzenia połączeń można uzyskać korzystając z poniższych poleceń

```
sizeof_collection [get_nets -hierarchical *]  
report_ignored_layers
```

Przed rozpoczęciem właściwego routingu wskazane jest sprawdzenie wykonalności tej operacji komendą:

```
check_routability
```

Operacja prowadzenie połączeń składa się z trzech etapów:

- Global routing
- Track assignment
- Detail routing

Każdy z powyższych etapów można wykonać osobno wywołując odpowiednio instrukcje: `route_global`, `route_track` oraz `route_detail`. Wszystkie trzy etapy można zrealizować łącznie komendą `route_auto`. Optymalizację prowadzenia połączeń zapewnia instrukcja `route_opt`. Ewentualne niewielkie naruszenia reguł projektowych możemy naprawić stosując komendę `route_eco`.

Aby zwiększyć niezawodność układu wskazane jest dodanie nadmiarowych przelotek, czemu służy instrukcja `add_redundant_vias`.

Po zakończeniu prowadzenia połączeń przeprowadzamy sprawdzenie poprawności wykonania tej operacji pod kontem naruszeń reguł projektowych i zgodności otrzymanego layoutu ze schematem logicznym układu. Do tego celu wykorzystujemy polecenia:

```
check_routes  
check_lvs
```

Sprawozdanie

W sprawozdaniu należy zamieścić informacje o najważniejszych parametrach układu po wykonaniu operacji routingu a także zrzuty ekranu pokazujące widok rdzenia układu po tej operacji - z włączoną i wyłączoną widocznością linii zasilania.

Ćwiczenie 5

Po rozmieszczeniu komórek standardowych i poprowadzeniu połączeń pozostają jeszcze do zrobienia pewien zestaw działań przygotowujących układ scalony ASIC do produkcji:

- wypełnienie pustych przestrzeni specjalnymi komórkami
- weryfikacja układu pod kątem spełniania wszystkich reguł projektowych i usunięcie drobnych przekroczeń tych reguł
- wypełnienie obszarów metalizacji
- zapisanie layotu w formacie GDS II
- zapisanie pozostałych informacji projektowych

Dostarczony skrypt `task5.tcl` realizuje powyżej wymienione zadania.

Sprawozdanie

W sprawozdaniu proszę zamieścić

- informacje o najważniejszych parametrach układu DUT po zakończeniu jego projektowania,
- końcowy widok layoutu układu.

UWAGA!

W sprawozdaniu mile widziane są również wszelkie spostrzeżenia, obserwacje i wnioski studentów. 😊